

SynPrompt: A Centralized AI Prompt Repository and Execution Management System

Contents

1 Problem Statement & Motivation	2
2 Objectives	2
3 System Architecture	2
4 Entity Relationship Diagram	3
5 Database Normalization	4
5.1 First Normal Form (1NF)	4
5.2 Second Normal Form (2NF)	4
5.3 Third Normal Form (3NF)	4
6 Database Schema Definitions	4
6.1 Users	4
6.2 Prompts	4
6.3 Projects	4
6.4 Categories	4
6.5 Project Prompts	5
6.6 Favorites	5
6.7 Models	5
6.8 Prompt Executions	5
7 Technology Stack	5
8 User Interface Implementation	5
8.1 Authentication Management	5
8.2 Main Dashboard View	6
8.3 Prompt Management	7
8.4 Project Management View	7
8.5 Execution History Log and Search functionality	8
8.6 Favorites Panel	9
9 Security Architecture & Data Validation	9
10 OpenRouter Multi-Model Integration	10
11 Development Challenges & Solutions	10
12 Future Roadmap Improvements	10

1 Problem Statement & Motivation

With the rapid growth of Artificial Intelligence (AI), developers, students, and researchers frequently use Large Language Models (LLMs) such as GPT, Gemini, and Claude to generate content, write code, summarize information, and solve complex problems. However, managing AI prompts across multiple platforms remains highly fragmented. Users often lose track of highly effective prompts, repeatedly rewrite similar intricate queries, and lack an organized system for cross-model comparisons and version tracking.

The objective of this project is to develop **SynPrompt**, an AI Prompt Repository that enables users to securely store, organize, execute, and manage AI prompts in a centralized environment. The system addresses these friction points by supporting structured project management, dynamic prompt categorization, custom favorites bookmarking, detailed execution history logs, and multi-model AI integration routed through the OpenRouter API.

2 Objectives

The core technical and functional objectives of SynPrompt are to:

- Provide secure user registration, session management, and password authentication.
- Implement full CRUD functionality allowing users to create, edit, delete, and organize AI prompts.
- Group related prompts into reusable, context-specific projects.
- Execute stored prompts across multiple disparate AI models seamlessly through a single OpenRouter API integration.
- Store generated AI responses locally within the database layer for decoupled future reference.
- Maintain a comprehensive, time-stamped execution history for performance auditing.
- Allow users to quickly bookmark and access frequent or high-performing prompts.
- Provide granular search and filtering functionality for prompts, categories, and projects.
- Demonstrate rigorous database normalization practices and relational integrity constraints.

3 System Architecture

The application architecture utilizes a traditional three-tier software engineering paradigm to enforce separation of concerns:

1. **Presentation Layer (Frontend):** Built using semantic HTML5 structures structured with Bootstrap 5 utility classes for a fully responsive grid system. Dynamic server-side rendering is handled via Python's Jinja2 templating engine.

2. **Application Layer (Backend):** Powered by the Flask micro-framework in Python. This layer contains the core business logic, handles routing, processes user inputs, performs authentication validations, and acts as the client proxy to the external OpenRouter API endpoint.
3. **Database Layer (Storage):** Implemented using a MySQL / MariaDB relational database server to manage structured entities, relational links, and referential integrity.

When an execution request occurs, the Flask application layer extracts the prompt text, securely appends context variables, routes the payload downstream to the OpenRouter gateway, captures the JSON return string, strips latency metrics, commits the text block cleanly into the database layer, and pipes the view back to the Presentation layer.

4 Entity Relationship Diagram

The logical database architecture consists of eight interdependent relational entities:

- Users • Prompts • Projects • Categories
- Project Prompts • Favorites • Models • Prompt Executions

Figure 1 illustrates the comprehensive Entity Relationship Diagram (ERD), charting logical constraints, primary key configurations, and relational cardinalities.

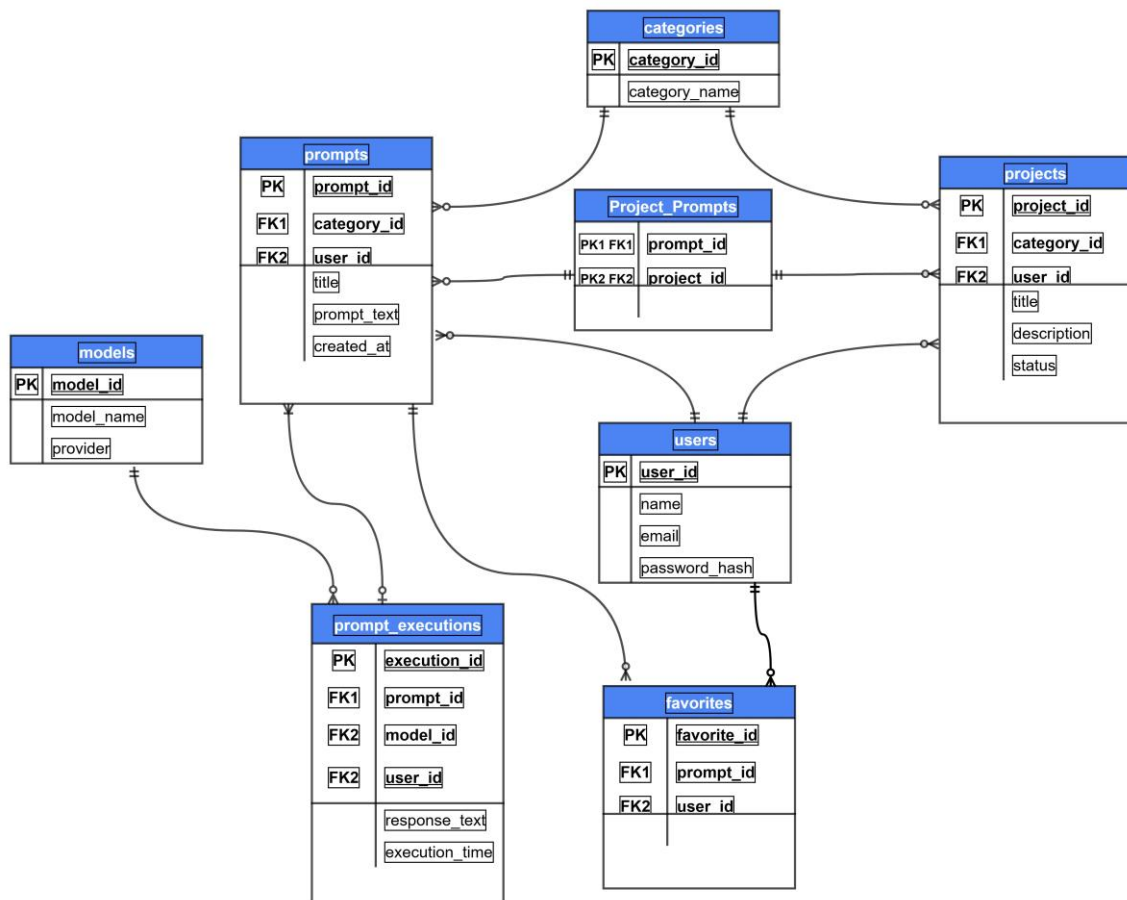


Figure 1: Entity Relationship Diagram for the SynPrompt Schema

5 Database Normalization

To eliminate logical redundancy and prevent anomalies during structural insertions, deletions, or data modifications, the backing relational architecture was iteratively normalized up to the Third Normal Form (3NF).

5.1 First Normal Form (1NF)

All tables contain strictly atomic values at every intersection of row and column. Repeating field blocks have been extracted, and each entity maintains an immutable, uniquely defined primary key index field.

5.2 Second Normal Form (2NF)

Each table satisfies all 1NF preconditions, and all non-key attributes exhibit full functional dependency on the entire primary key. Partial composite dependencies are eliminated; specifically, the many-to-many relationship mapping between Projects and Prompts is cleanly resolved using a dedicated associative/junction entity table named Project _Prompts.

5.3 Third Normal Form (3NF)

Transitive dependencies have been thoroughly removed. Non-prime fields do not depend on other non-prime fields via a transitive path. Lookup attributes, such as independent model details and descriptive categories, have been extracted into isolated tables (Models and Categories), establishing rigid referential integrity via structural foreign key constraints.

6 Database Schema Definitions

The structural definition of the implemented database consists of the following foundational tables:

6.1 Users

Stores system credential details including unique auto-incrementing identifiers (user id), alphanumeric descriptive strings (name, email), and secure cryptographic strings (password hash).

6.2 Prompts

The core content storage unit holding specific query blueprints. It maps a unique identifier (prompt id) back to an owner (user _id) and an organized category metadata tag (category id), alongside contextual fields (title, prompt text, created _at).

6.3 Projects

Allows logical groupings of functional objectives. Captures user references, project titles, text descriptions, and an operational lifecycle status flag (status).

6.4 Categories

A standardized system lookup table providing taxonomy labels (category _name) linked to specific data rows via a foreign identifier mapping (category id).

6.5 Project Prompts

An associative bridge table managing the resolved composite key structures (prompt id and project id) required to handle complex cross-table indexing without duplication.

6.6 Favorites

Tracks user engagement by capturing individual instances where a unique user bookmarks a particular template entry for instantaneous retrieval.

6.7 Models

An immutable catalogue tracking global multi-vendor endpoint information including structural API tags (model _id), public commercial names (model name), and parent organizations (provider).

6.8 Prompt Executions

Logs empirical transactions. Connects an executed template to a designated LLM endpoint, tracking the exact response string data block alongside the operational resource overhead (execution time).

7 Technology Stack

The selected technologies provide a lightweight, scalable, and maintainable architecture suitable for a web-based database management system.

Technology	Operational Purpose
Python	Backend Scripting and Runtime Architecture
Flask	Micro-web Application Server and Routing Framework
HTML5	Semantic Markup and Structural Presentation
Bootstrap 5	Layout Engineering and Component Utility Styling
Jinja2	Server-Side String Interpolation and Template Interactivity
MySQL / MariaDB	Relational Engine for Relational Data Persistence
OpenRouter API	Agnostic Multi-LLM Cloud Execution Route Gateway

Table 1: Architectural Technology Stack Overview

8 User Interface Implementation

The presentation layout exposes interactive views optimized for administrative overhead reduction and developer ease of use.

8.1 Authentication Management

The portal restricts access behind standard verification forms. Both layouts utilize centered card interfaces with clean input validation formatting to ensure secure data transmission before user state authorization.

The login form is titled "Login". It contains two input fields: "Email" and "Password". Below the "Password" field is a green "Login" button. At the bottom of the form is a blue link that says "Create Account".

Figure 2: Portal Login Interface

The "Create Account" form is titled "Create Account". It contains three input fields: "Name", "Email", and "Password". Below the "Password" field is a blue "Register" button. At the bottom of the form is a blue link that says "Already have an account?".

Figure 3: User Registration Form

Figure 4: User Portal Authentication Framework Screens

8.2 Main Dashboard View

Displays foundational, real-time user metrics including total compiled prompts, running active projects, aggregated bookmark favorites, and a historical execution counter widget. The dynamic Multi-Model Routing Hub allows swift selection across active LLM backends.

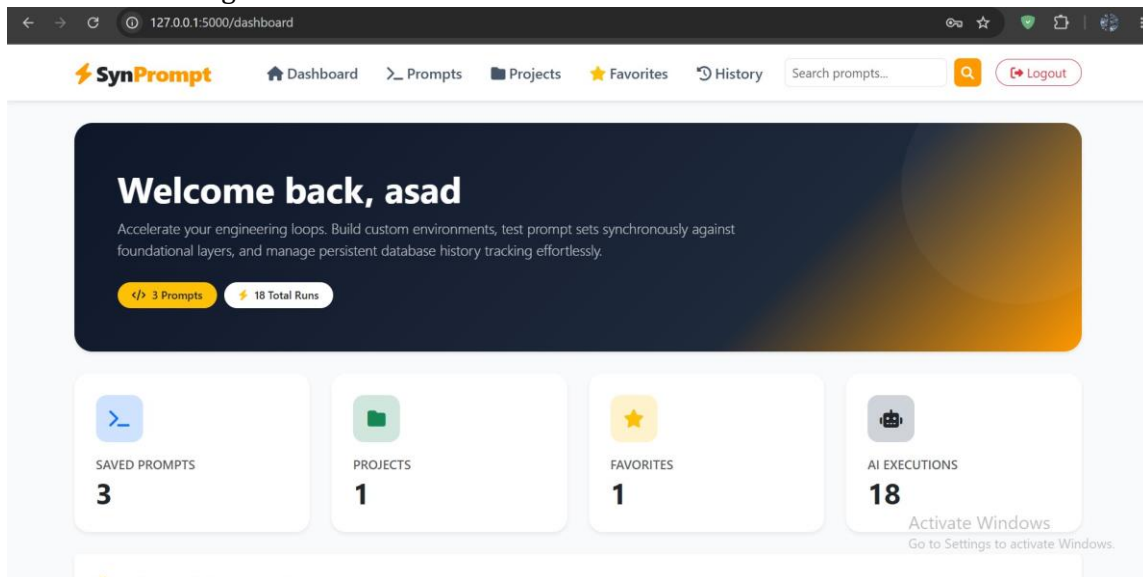


Figure 5: Main Dashboard Analytics and Summary Cards

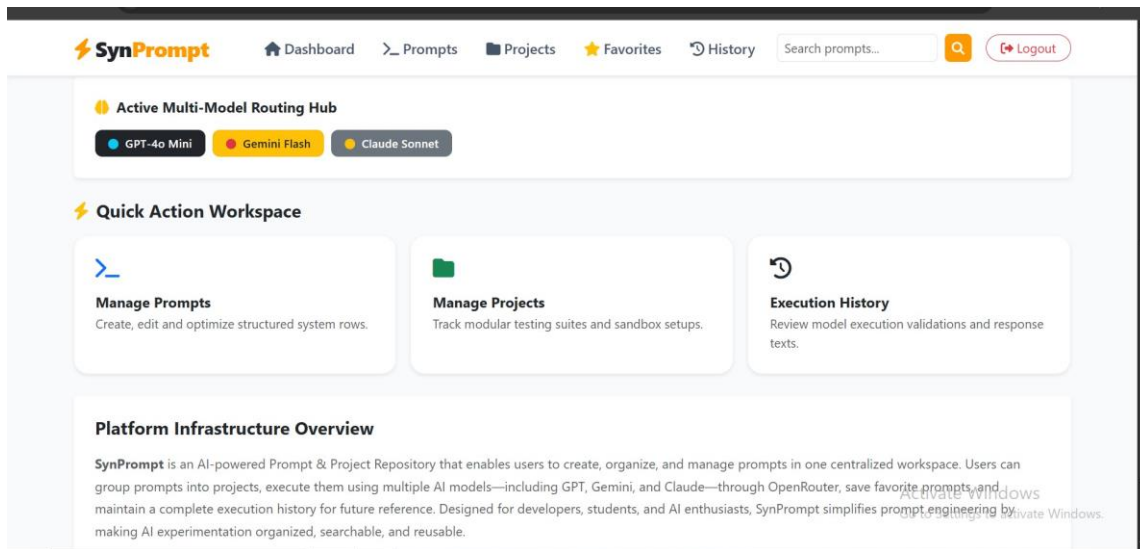


Figure 6: Dashboard Multi-Model Hub and Quick Action Workspaces

8.3 Prompt Management

Provides intuitive interactive tables containing actionable links allowing users to rapidly execute full CRUD lifecycle operations on raw prompt records. Action elements include inline options to edit, delete, favorite, or run individual templates immediately.

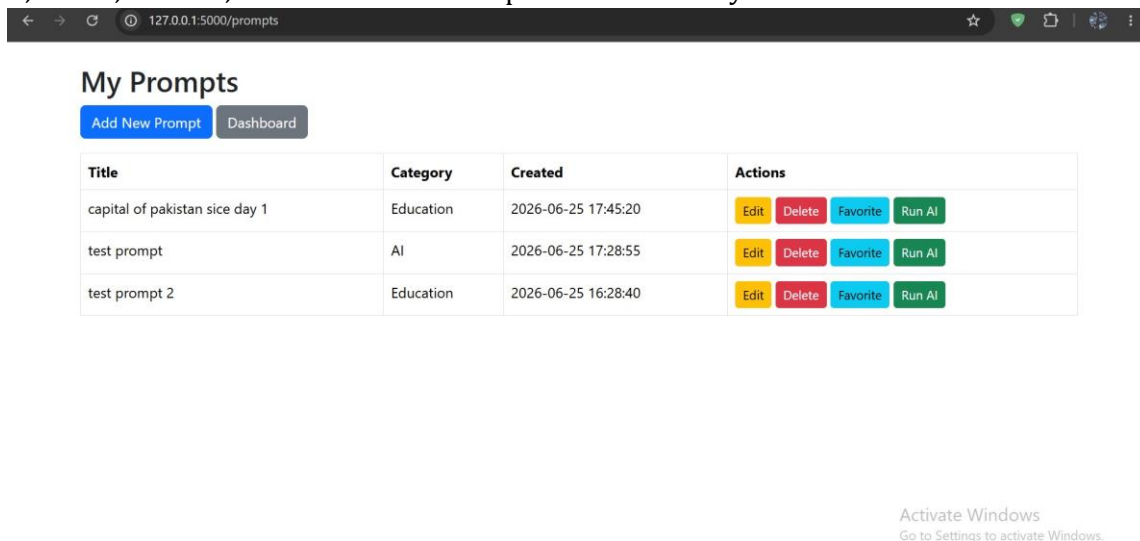


Figure 7: Prompt Library and Action Control Interface

8.4 Project Management View

Exposes layout mechanisms allowing developers to collectively gather separate context prompts into execution batches, facilitating workflow pipeline automation and categorization by operational status.

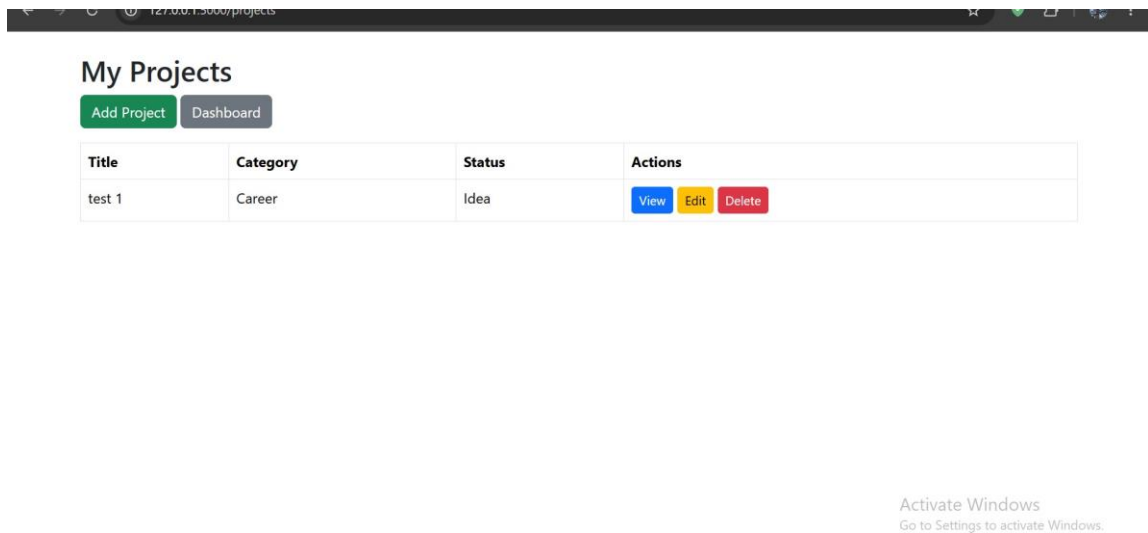


Figure 8: Project Workspaces and Execution Pipelines

8.5 Execution History Log and Search functionality

The logging module provides an immutable historical dashboard display outputting structured data detailing previous prompt strings, active vendor model tags, operational timestamps, and exact response text generation blocks. Granular querying is handled seamlessly through the system search results view.

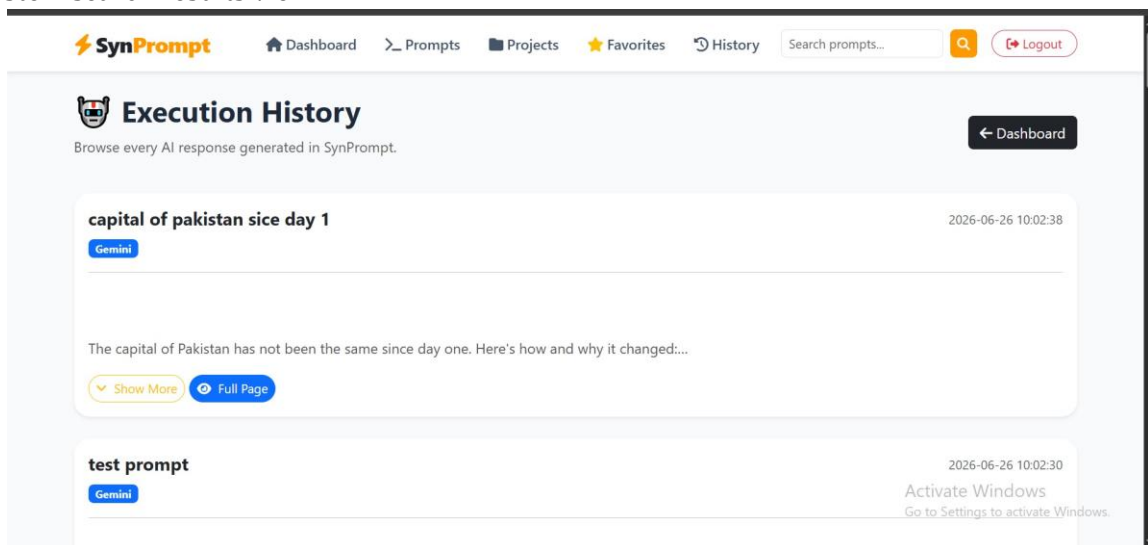


Figure 9: Immutable AI Execution History Log

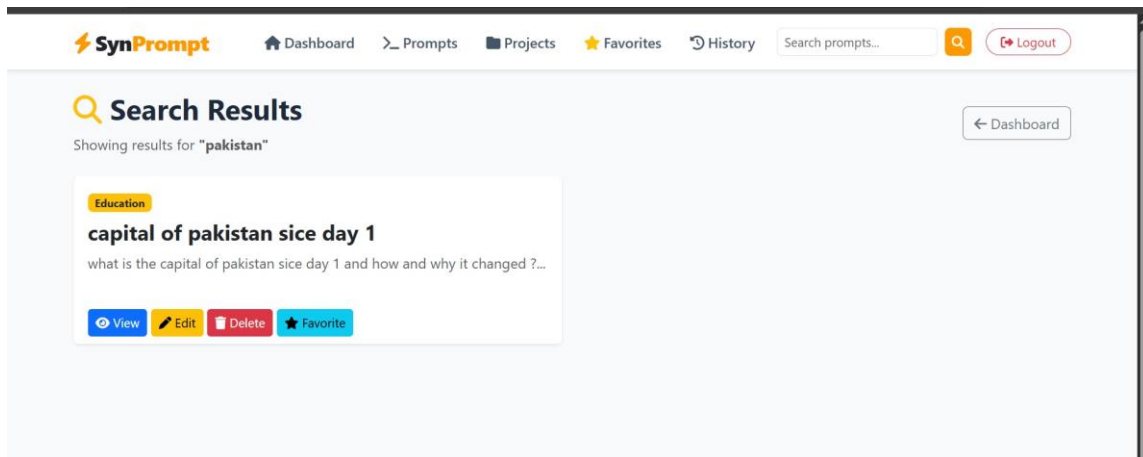


Figure 10: Granular Global Search Results View

8.6 Favorites Panel

A minimalist, high-speed layout designed for instant visibility, allowing single-click retrieval and optimization of heavily recycled prompt structures.

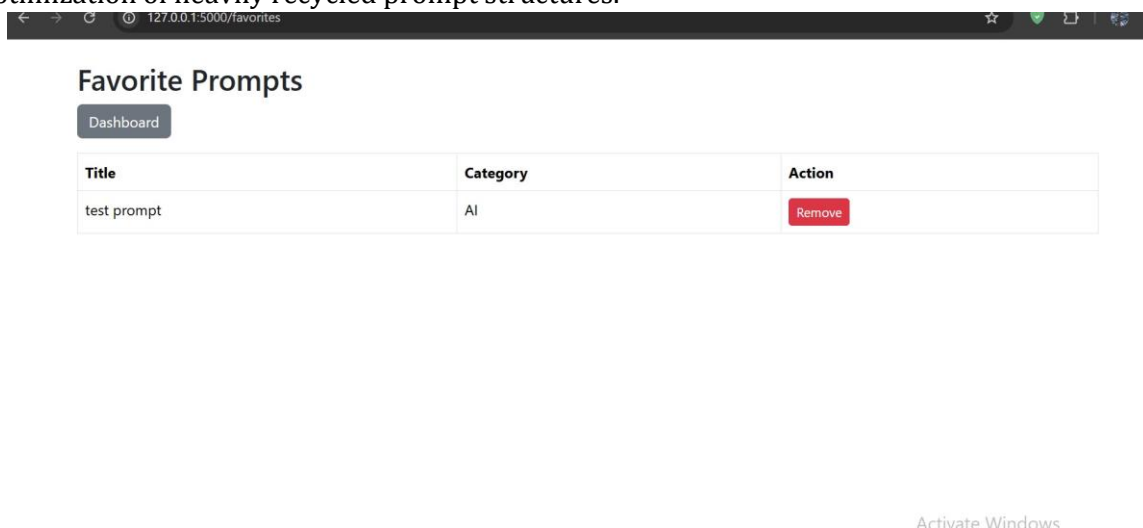


Figure 11: Bookmarked Favorites Quick-Access Grid

9 Security Architecture & Data Validation

To meet enterprise compliance standards for data management, several application safety parameters are strictly maintained:

- **Cryptographic Hashing:** Text credentials are never saved as raw text. The backend processes inbound registration payloads through a strong salt routing mechanism using the bcrypt work-factor adaptation scheme.
- **State Management:** User authentication boundaries are reinforced server-side using secure, cryptographically signed HTTP cookies managed via native Flask session client states.

- **Injection Mitigation:** Raw queries avoid string formatting operations. Database interfaces execute parameters exclusively through transactional object-relational abstraction parameters to isolate execution code from string tokens.
- **Route Guarding:** Access decorators evaluate state profiles prior to delivering specific JSON views or structural updates, blocking unauthenticated URL navigation.

10 OpenRouter Multi-Model Integration

Rather than coupling the system directly to isolated corporate endpoints, SynPrompt leverages an asynchronous gateway connection layer via the OpenRouter API proxy. This provides structural poly-model interaction capabilities using a normalized JSON communication payload format.

The application actively interfaces with the following production models:

- **GPT-4o Mini:** Tailored for rapid, high-throughput prompt analysis and structural data cleanup.
- **Google Gemini:** Leveraged for tasks requiring massive context scanning and creative problem extraction.
- **Anthropic Claude:** Utilized for highly complex programmatic evaluations and strict logical constraints.

11 Development Challenges & Solutions

Throughout development, the engineering workflow presented multiple technical hurdles requiring tactical remediation:

- **Schema Normalization Balancing:** Translating loosely coupled textual workflows into rigid transactional entities required several iterative revisions before completely eliminating multi-value field attributes without disrupting application throughput.
- **Many-to-Many Resolution Management:** Configuring stable indexing across associative entities without generating orphaned rows required writing explicit cascading constraints into the SQL schema.
- **API Failure Engineering:** Unreliable external networks or deprecated cloud endpoint paths could freeze application threads. This was solved by designing a centralized errorinterception module that logs errors and falls back to cached prompt states without crashing the user session.

12 Future Roadmap Improvements

The operational capacity of SynPrompt can be augmented further through the following targeted updates:

- **Parallel Execution Analytics:** Simultaneous distribution of a single prompt text across multiple models to display real-time comparative metrics.

- **Prompt Versioning Control:** Implementing chronological delta version branches allowing rollback states for prompt texts.
- **Collaborative Shared Workspaces:** Introducing localized tenancy levels to let development teams co-author prompt libraries in shared project domains.
- **Automated Document Exporters:** Generating programmatic reports that export execution histories cleanly into standardized PDF summaries or raw analytical CSV data dumps.